The Logic of Hebbian Learning

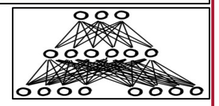
Caleb Kisby, Saúl Blanco, Larry Moss

Introduction

Symbolic Systems
- ✓ Sophisticated rich reasoning
- ✓ Explainable decisions
- ✗ Notoriously rigid and static
- ✗ Manual knowledge-engineering

Neural Networks
- ✗ Can't readily learn rich inference
- ✗ "Black Box" decisions
- ✓ Learns from experience
- ✓ Uses unstructured data

penguin → bird
bird ⇒ flies
¬(bird ∧ penguin ⇒ flies)



Problem:
How do we bridge the two?

Proposal:
The neural network forms the semantics that we use to interpret the logic

Propagation and Hebbian Update

Neural networks are given by $\mathcal{N} = \langle N, E, W, A, O, \eta \rangle$:

N	Set of neurons	$W : N \times N$	Weights
E	Excitatory connections	$A^{(n)} : \mathbb{R}^k \to \mathbb{R}$	Activation function
$\eta \in \mathbb{R}$	Learning rate	$O^{(n)} : \mathbb{R} \to \{0,1\}$	Output function

- **Feedforward:** E has no cycles with all nonzero weights
- **Binary:** $O^{(n)}(x) \in \{0,1\}$
- **Monotonic:** $A^{(n)}(\vec{x})$ is monotonically increasing (this includes sigmoid functions)

Definition 2. Let $\mathsf{Prop} : \mathsf{Set} \to \mathsf{Set}$; $n \in \mathsf{Prop}(S)$ iff either
(Base Case) $n \in S$, or
(Constructor) For those $m_1, \ldots, m_k \in \mathsf{Prop}(S)$ such that $(m_i, n) \in E$ we have $O^{(n)}(A^{(n)}(\overrightarrow{W}(m_i, n))) = 1$

Definition 4. Let $\mathsf{Inc} : \mathsf{Net} \times \mathsf{Set} \to \mathsf{Net}$ be given by $\mathsf{Inc}(\langle N, E, W, A, O, \eta \rangle, S) = \langle N, E, W^*, A, O, \eta \rangle$, where

$$W^*_{ij} = W_{ij} + \eta \cdot \chi_{\mathsf{Prop}(S)}(i) \cdot \chi_{\mathsf{Prop}(S)}(j)$$

The Neuro-Symbolic Interface

$$\varphi ::= p \mid \top \mid \neg\varphi \mid \varphi \wedge \varphi \mid \varphi \to \varphi \mid \varphi \Rightarrow \varphi \mid \mathbf{T}\varphi \mid [\varphi^+]\varphi$$

$$\begin{array}{lcl}
[\![p]\!] & & \in \mathsf{Set} \text{ is fixed, nonempty} \\
[\![\top]\!] & = & \emptyset \\
[\![\neg\varphi]\!] & = & \overline{[\![\varphi]\!]} \\
[\![\varphi \wedge \psi]\!] & = & [\![\varphi]\!] \cup [\![\psi]\!] \\
[\![\varphi \to \psi]\!] & = & [\![\top]\!] \text{ iff } [\![\varphi]\!] \supseteq [\![\psi]\!], \text{ else } [\![\bot]\!] \\
[\![\varphi \Rightarrow \psi]\!] & = & [\![\top]\!] \text{ iff } \mathsf{Prop}([\![\varphi]\!]) \supseteq [\![\psi]\!], \text{ else } [\![\bot]\!] \\
[\![\mathbf{T}\varphi]\!] & = & \mathsf{Prop}([\![\varphi]\!]) \\
[\![[\varphi^+]\psi]\!] & = & [\![\psi]\!]_{\mathsf{Inc}(\mathcal{N}, [\![\varphi]\!])}
\end{array}$$

The Rules of Hebbian Learning

Basic Axioms

(PC) All proposotional tautologies

(DUAL) $\langle \mathbf{T} \rangle \varphi \leftrightarrow \neg \mathbf{T} \neg \varphi$ (T) $\mathbf{T}\varphi \to \varphi$

(N) $\mathbf{T}\top$ (4) $\mathbf{T}\varphi \to \mathbf{T}\mathbf{T}\varphi$

Inference Rules

(MP) $\dfrac{\varphi \quad \varphi \to \psi}{\psi}$

(TYP) $\dfrac{\varphi \Rightarrow \psi}{\mathbf{T}\varphi \to \psi}$ $\dfrac{\mathbf{T}\varphi \to \psi}{\varphi \Rightarrow \psi}$ (NEC$_+$) $\dfrac{\psi}{[\varphi^+]\psi}$

(C$_\Rightarrow$) $\dfrac{\varphi \to \psi \quad \psi \Rightarrow \varphi}{\varphi \Leftrightarrow \psi}$ (C$_+$) $\dfrac{\psi \to \rho \quad [\varphi^+]\rho \to \psi}{[\varphi^+]\psi \leftrightarrow [\varphi^+]\rho}$

(LOOP$_+$) $\dfrac{[\varphi^+]\psi_0 \to \psi_1 \cdots [\varphi^+]\psi_{k-1} \to \psi_k \quad [\varphi^+]\psi_k \to \psi_0}{[\varphi^+]\psi_0 \to \psi_k}$

(LOOP$_\Rightarrow$) $\dfrac{\varphi_0 \Rightarrow \varphi_1 \cdots \varphi_{k-1} \Rightarrow \varphi_k \quad \varphi_k \Rightarrow \varphi_0}{\varphi_0 \Rightarrow \varphi_k}$

Reduction Axioms

(R$_p$) $[\varphi^+]p \leftrightarrow p$ (R$_\neg$) $[\varphi^+]\neg\psi \leftrightarrow \neg[\varphi^+]\psi$

(R$_\wedge$) $[\varphi^+](\psi \wedge \rho) \leftrightarrow ([\varphi^+]\psi \wedge [\varphi^+]\rho)$

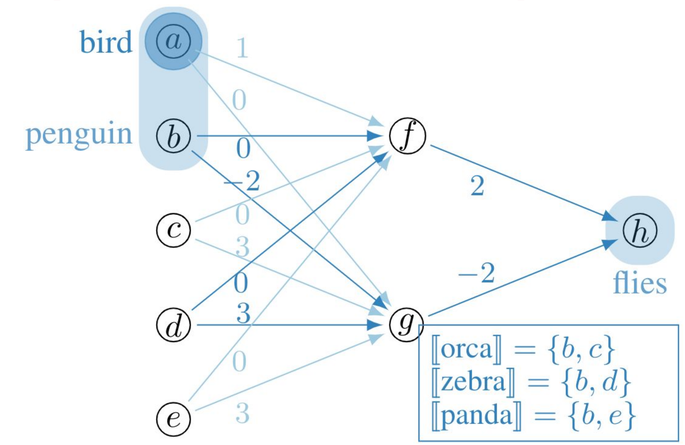
(NEST$_\mathbf{T}$) $[\mathbf{T}\varphi^+]\psi \leftrightarrow [\varphi^+]\psi$

Key Axioms

(NS) $[\varphi^+]\mathbf{T}\psi \to \mathbf{T}[\varphi^+]\psi$

(TP) $\mathbf{T}[\varphi^+]\psi \wedge \mathbf{T}\varphi \to [\varphi^+]\mathbf{T}\psi$

Implementation & Example




```
nodes = set(['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h'])
layers = [['a', 'b', 'c', 'd', 'e'], ['f', 'g'], ['h']]
weights = {('a', 'f'): 1.0, ('a', 'g'): 0.0,
           ('b', 'f'): 0.0, ('b', 'g'): -2.0,
           ('c', 'f'): 0.0, ('c', 'g'): 3.0,
           ('d', 'f'): 0.0, ('d', 'g'): 3.0,
           ('e', 'f'): 0.0, ('e', 'g'): 3.0,
           ('f', 'h'): 2.0, ('g', 'h'): -2.0}
threshold = 0.0
rate = 1.0
prop_map = {'bird': {'a'}, 'penguin': {'a', 'b'},
            'orca': {'b', 'c'}, 'zebra': {'b', 'd'},
            'panda': {'b', 'e'}, 'flies': {'h'}}
net = BFNN(nodes, layers, weights, threshold, rate)
model = Model(net, prop_map)
```

```
> penguin → bird
    True

> bird ⇒ flies
    True

> penguin ⇒ flies
    True

> orca+ zebra+ panda+
>  (bird ⇒ flies)
    True

> orca+ zebra+ panda+
>  (penguin ⇒ flies)
    False
```

Acknowledgements

For references, see our FLAIRS-35 conference paper.
(Just ask me for a copy!)
This work was funded by the US Department of Defense (Contract W52P1J2093009). Thank you for your support!

LUDDY SCHOOL OF INFORMATICS, COMPUTING, AND ENGINEERING